

CLAUDE DEVELOPMENT

Practical Course Handbook

Models, API usage, prompting, evaluation, tools, RAG, MCP, Claude Code, and agent workflows

A structured study guide distilled from the complete course notes

Reference Edition

Quick Reference

Use this section to choose a model, understand the API flow, and locate the major course themes quickly.

Model	Priority	Best Fit	Trade-off
Opus	Intelligence	Complex reasoning, planning, multi-step work	Higher cost and latency
Sonnet	Balance	Most practical tasks; coding and precise edits	Middle ground
Haiku	Speed	Real-time interactions and high-volume processing	Lower reasoning depth

API request essentials: Keep the API key on the server. Send model, messages, and max_tokens. The response includes generated content, token usage, and stop_reason.

Conversation memory: The API is stateless. Preserve and resend the complete message history for context-aware multi-turn conversations.

Prompt quality: Start with a direct action verb, specify output attributes and steps, structure interpolated content with XML tags, and use examples for difficult formats or edge cases.

Reliability first: Use prompt evaluations, code/model graders, workflows, and syntax validators rather than relying on one or two manual tests.

Tools and agents: Tools extend Claude with live data and actions. Prefer reliable workflows when steps are known; use agents when the path must be planned dynamically.

Contents

1. Foundations and API Basics
2. Prompt Evaluation and Prompt Engineering
3. Tool Use and Structured Outputs
4. Built-in Tools, Search, RAG, and Multimodal Input
5. Performance, Files, and Code Execution
6. Model Context Protocol (MCP)
7. Claude Code and Computer Use
8. Agents and Workflow Patterns

Each module follows the original course sequence and preserves the terminology and implementation patterns introduced in the lessons.

MODULE 1

Foundations and API Basics

Overview of Claude Models

Claude has three model families optimized for different priorities:

Opus - highest intelligence model for complex, multi-step tasks requiring deep reasoning and planning. Trade-off: higher cost and latency.

Sonnet - balanced model with good intelligence, speed, and cost efficiency. Strong coding abilities and precise code editing. Best for most practical use cases.

Haiku - fastest model optimized for speed and cost efficiency. No reasoning capabilities like Opus/Sonnet. Best for real-time user interactions and high-volume processing.

Selection framework: Intelligence priority -> Opus. Speed priority -> Haiku. Balanced requirements -> Sonnet.

Common approach - use multiple models in same application based on specific task requirements rather than single model selection.

All models share core capabilities: text generation, coding, image analysis. Main difference is optimization focus.

Accessing the API

API Access Flow - 5-step process from user input to response display

Step 1: Client sends user text to developer's server (never access Anthropic API directly from client apps to keep API key secret)

Step 2: Server makes request to Anthropic API using SDK (Python, TypeScript, JavaScript, Go, Ruby) or plain HTTP. Required parameters - API key + model name + messages list + max_tokens limit

Step 3: Text generation process has 4 stages:

- Tokenization = breaking input into tokens (words/word parts/symbols/spaces)
- Embedding = converting tokens to number lists representing all possible word meanings
- Contextualization = adjusting embeddings based on neighboring tokens to determine precise meaning
- Generation = output layer produces probabilities for next word, model selects using probability + randomness, adds selected word, repeats process

Step 4: Model stops when max_tokens reached or special end_of_sequence token generated

Step 5: API returns response with generated text + usage counts + stop_reason to server, server sends to client for display

Token - text chunk (word/part/symbol)

Embedding - numerical representation of word meanings

Contextualization - meaning refinement using neighboring words

Max_tokens - generation length limit

Stop_reason - why model stopped generating

Making a Request

Making API Request to Anthropic - Process involving 4 setup steps and understanding message structure

Setup Steps:

9. Install packages = pip install anthropic python-dotenv in Jupyter notebook
10. Store API key = Create .env file with ANTHROPIC_API_KEY="your_key" (ignore in version control)
11. Load environment variable = Use python-dotenv to securely load API key
12. Create client = Initialize anthropic client and define model variable (claude-3-sonnet)

API Request Structure:

- Function = client.messages.create()
- Required arguments = model, max_tokens, messages
- Model = Name of Claude model to use
- Max_tokens = Safety limit for generation length (not target length)
- Messages = List containing conversation exchanges

Message Types:

- User message = {"role": "user", "content": "your text"} (human-authored content)
- Assistant message = Contains model-generated responses

Response Access:

- Full response = Contains metadata and nested structure
- Text only = message.content[0].text extracts just generated text

Example request structure: client.messages.create(model=model, max_tokens=1000, messages=[{"role": "user", "content": "What is quantum computing?"})

Multi-Turn Conversations

Multi-Turn Conversations - conversations with multiple back-and-forth exchanges that maintain context.

Key limitation: Anthropic API stores no messages. Each request is independent with no memory of previous exchanges.

Solution requires two steps:

13. Manually maintain message list in code
14. Send entire conversation history with every follow-up request

Message structure - list of dictionaries with "role" (user/assistant) and "content" fields.

Conversation flow:

- Send initial user message
- Receive assistant response
- Append assistant response to message history
- Add new user message to history

- Send complete history for context-aware follow-up

Helper functions needed:

- `add_user_message(messages, text)` = appends user message to history
- `add_assistant_message(messages, text)` = appends assistant response to history
- `chat(messages)` = sends message history to API and returns response

Without message history - responses lack context and continuity. With complete history = Claude maintains conversation context and provides relevant follow-ups.

System Prompts

System Prompts - technique to customize Claude's response style and tone by assigning it a specific role or behavior pattern.

Implementation - pass system prompt as plain string to create function using system keyword argument.

Purpose - control how Claude responds rather than what it responds. Example: math tutor role makes Claude give hints instead of direct answers.

Structure - first line typically assigns role ("You are a patient math tutor"), followed by specific behavioral instructions.

Key principle - system prompts guide response approach, not content. Same question gets different treatment based on assigned role.

Technical implementation - create params dictionary, conditionally add system key if prompt provided, pass params to create function with unpacking. Handle None case by excluding system parameter entirely.

Use case example - Math tutor that gives guidance/hints rather than complete solutions, encouraging student thinking over direct answers.

Temperature

Temperature - parameter (0-1) that controls randomness in Claude's text generation by influencing token selection probabilities.

Text generation process: Input text -> tokenization -> probability assignment to possible next tokens -> token selection based on probabilities -> repeat.

Temperature effects:

- Temperature 0 = deterministic output, always selects highest probability token
- Higher temperature = increases chance of selecting lower probability tokens, more creative/unexpected outputs

Usage guidelines:

- Low temperature (near 0) = data extraction, factual tasks requiring consistency
- High temperature (near 1) = creative tasks like brainstorming, writing, jokes, marketing

Implementation: Add temperature parameter to model API calls. Higher values don't guarantee different outputs, just increase probability of variation.

Key insight: Temperature directly manipulates the probability distribution of next token selection, making high-probability tokens more/less dominant in the selection process.

Response Streaming

Response Streaming - technique to display AI responses chunk-by-chunk as they're generated instead of waiting for complete response.

Problem solved: AI responses can take 10-30 seconds. Users expect immediate feedback, not just spinners.

How it works:

15. Server sends user message to Claude
16. Claude immediately sends initial response (no text, just acknowledgment)
17. Stream of events follows, each containing text chunks
18. Server forwards chunks to frontend for real-time display

Event types:

- `message_start` = initial acknowledgment
- `content_block_start` = text generation begins
- `content_block_delta` = contains actual text chunks (most important)
- `content_block_stop/message_stop` = generation complete

Implementation:

Basic: `client.messages.create(stream=True)` returns event iterator

Simplified: `client.messages.stream()` with `text_stream` property extracts just text

Final message: `stream.get_final_message()` assembles all chunks for storage

Key benefits: Better UX through immediate response visibility, complete message capture for database storage.

Controlling Model Output

Controlling Model Output = Two key techniques beyond prompt modification

Pre-filling Assistant Messages = Manually adding assistant message at end of conversation to steer response direction

How it works:

- Assemble messages list with user prompt + manual assistant message
- Claude sees assistant message as already authored content
- Claude continues response from exact end of pre-filled text
- Response gets steered toward pre-filled direction

Key point: Claude continues from exact endpoint of pre-fill, not complete sentences. Must stitch together pre-fill + generated response.

Example: Pre-fill "Coffee is better because" -> Claude continues with justification for coffee

Stop Sequences = Force Claude to halt generation when specific string appears

How it works:

- Provide stop sequence string in chat function
- When Claude generates that exact string, response immediately stops
- Generated stop sequence text not included in final output

Example: Prompt "count 1 to 10" + stop sequence "five" -> Output stops at "four, " (five not included)

Refinement: Stop sequence ", five" -> Clean output "one, two, three, four"

Both techniques provide precise control over response direction and length without changing core prompts.

Structured Data

Structured Data Generation - technique using assistant message prefilling + stop sequences to get raw output without Claude's natural explanatory headers/footers.

Problem - Claude automatically adds markdown formatting, headers, commentary when generating JSON/code/structured content. Users often want just the raw data for copy/paste functionality.

Solution Pattern:

19. User message = request for structured data
20. Assistant message prefill = opening delimiter (e.g., "\\json")
21. Stop sequence = closing delimiter (e.g., "\\")

How it works - Claude sees prefilled message, assumes it already started response, generates only the requested content, stops when hitting delimiter.

Result - Raw structured data output with no extra formatting or commentary.

Application - Works for any structured data type (JSON, Python code, lists, etc.), not just JSON. Use whenever you need clean, parseable output without explanatory text.

Key benefit - Output can be directly used/copied without manual selection or parsing of unwanted text.

MODULE 2

Prompt Evaluation and Prompt Engineering

Prompt Evaluation

Prompt Engineering - techniques for writing/editing prompts to help Claude understand requests and desired responses.

Prompt Evaluation - automated testing of prompts using objective metrics to measure effectiveness.

Three paths after writing a prompt:

- 22. Test once/twice, deploy to production (trap)
- 23. Test with custom inputs, minor tweaks for corner cases (trap)
- 24. Run through evaluation pipeline for objective scoring (recommended)

Key takeaway: Engineers commonly under-test prompts. Use evaluation pipelines to get objective performance scores before iterating and deploying prompts.

A Typical Eval Workflow

Typical Eval Workflow - 6-step iterative process for prompt improvement

Step 1: Write initial prompt draft - create baseline prompt to optimize

Step 2: Create evaluation dataset - collection of test inputs (can be 3 examples or thousands, hand-written or LLM-generated)

Step 3: Generate prompt variations - interpolate each dataset input into prompt template

Step 4: Get LLM responses - feed each prompt variation to Claude, collect outputs

Step 5: Grade responses - use grader system to score each response (e.g. 1-10 scale), average scores for overall prompt performance

Step 6: Iterate - modify prompt based on scores, repeat entire process, compare versions

Key points: No standard methodology exists. Many open-source/paid tools available. Can start simple with custom implementation. Grading complexity varies. Objective scoring enables systematic prompt improvement through A/B comparison.

Generating Test Datasets

Custom prompt evaluation workflow - build prompt + generate test dataset + evaluate performance

Goal - AWS code assistance prompt that outputs only Python, JSON config, or regex without explanations

Dataset generation approaches - manual assembly or automated with Claude (use faster models like Haiku for generation)

Dataset structure - array of JSON objects with task property describing user requests

Generation process - prompt Claude to create test cases -> use pre-filling with assistant message "\\json" -> set stop sequence "\\n" -> parse response as JSON -> save to file

Key implementation - generate_dataset() function that sends prompt to Claude, gets structured JSON response of test tasks, saves to dataset.json file for later evaluation use

Test dataset enables systematic evaluation by running prompt against multiple input scenarios to measure performance consistency.

Running the Eval

Eval execution process - merging test cases with prompts, running through LLM, and grading outputs.

Test case - individual record from dataset (JSON object).

Three core functions:

- run_prompt = merges test case with prompt, sends to Claude, returns output
- run_test_case = calls run_prompt, grades result, returns summary dictionary
- run_eval = loops through dataset, calls run_test_case for each, assembles results

Basic prompt structure - "Please solve the following task: [test_case_task]" (v1 starting point).

Current limitations - no output formatting instructions, hardcoded scoring (score=10), verbose Claude responses.

Runtime - ~31 seconds with Haiku model for full dataset execution.

Output format - array of objects containing Claude output, original test case, and score.

Next step - implement proper grading system to replace hardcoded scores.

Eval pipeline core - dataset + prompt + LLM + grader, with minimal code complexity.

Model Based Grading

Model Based Grading - evaluation system that takes model outputs and assigns objective scores (typically 1-10 scale, 10 = highest quality)

Three grader types:

- Code graders = programmatic checks (length, word presence, syntax validation, readability scores)
- Model graders = additional API call to evaluate original model output, highly flexible for quality/instruction-following assessment
- Human graders = person evaluates responses, most flexible but time-consuming and tedious

Key requirements: Must return objective signal (usually numerical score). Define evaluation criteria upfront.

Implementation pattern for model graders:

- Create detailed prompt requesting strengths/weaknesses/reasoning/score (not just score alone to avoid default middling scores)
- Use JSON response format with pre-filled assistant message and stop sequences
- Parse returned JSON for score and reasoning
- Calculate average scores across test cases for final metric

Model graders offer high flexibility but may be inconsistent. Still provides objective baseline for prompt optimization.

Code Based Grading

Code Based Grading - automated validation system for LLM outputs containing code, JSON, or regex

Core Implementation:

- `validate_json()` = attempts JSON parsing, returns 10 if valid, 0 if error
- `validate_python()` = attempts AST parsing, returns 10 if valid, 0 if error
- `validate_regex()` = attempts regex compilation, returns 10 if valid, 0 if error

Dataset Requirements:

- Must include "format" key specifying expected output type (JSON/Python/RegEx)
- Updated via prompt template modification for automated dataset generation

Prompt Engineering:

- Instruct model to respond only with raw code/JSON/regex
- No comments, explanations, or commentary
- Use pre-filled Assistant message with `\\code\\` blocks
- Add stop sequences to extract clean output

Scoring System:

- Final score = $(\text{model_score} + \text{syntax_score}) / 2$
- Combines semantic evaluation with syntax validation
- Enables measurement of both correctness and technical validity

Key Limitation - requires known expected format for proper validator selection

Prompt Engineering

Prompt Engineering - improving prompts to get more reliable, higher-quality outputs from language models.

Module Structure: Start with initial poor prompt -> Apply prompt engineering techniques step-by-step -> Evaluate improvements after each technique -> Observe performance gains over time.

Example Goal: Generate one-day meal plan for athletes based on height, weight, physical goal, dietary restrictions.

Technical Setup:

- Updated eval pipeline with flexible prompt evaluator class
- Supports concurrency (adjust `max_concurrent_tasks` based on rate limits)
- `generate_dataset()` method creates test cases with specified inputs
- `run_prompt()` function processes each test case individually

Key Components:

- `prompt_input_spec` = dictionary defining required prompt inputs
- `extra_criteria` = additional validation requirements for model grading
- `output.html` = formatted evaluation report showing test case results and scores

Process: Write initial prompt -> Interpolate test case inputs -> Run evaluation -> Apply engineering techniques -> Re-evaluate -> Repeat until satisfactory performance.

Initial Results: Expect poor scores (example: 2.32) with basic prompts, especially when using less capable models. Scores improve as techniques are applied.

Being Clear and Direct

Being Clear and Direct - Use simple, direct language with action verbs in the first line of prompts to specify the exact task.

First line importance - Most critical part of prompt that sets the foundation for AI response.

Structure - Action verb + clear task description + output specifications.

Examples:

- "Write three paragraphs about how solar panels work"
- "Identify three countries that use geothermal energy and for each include generation stats"
- "Generate a one day meal plan for an athlete that meets their dietary restrictions"

Key components - Action verb at start + direct task statement + expected output details.

Result - Improved prompt performance (example showed score increase from 2.32 to 3.92).

Being Specific

Being Specific - adding guidelines or steps to direct model output in particular direction

Two types of guidelines:

Type A (Attributes) - list qualities/attributes desired in output (length, structure, format)

Type B (Steps) - provide specific steps for model to follow in reasoning process

Type A controls output characteristics. Type B controls how model arrives at answer.

Both techniques often combined in professional prompts.

When to use:

- Type A (attributes): recommended for almost all prompts
- Type B (steps): use for complex problems where you want model to consider broader perspective or additional viewpoints it might not naturally consider

Example improvement: meal planning prompt score jumped from 3.92 to 7.86 when guidelines added, demonstrating significant quality improvement through specificity.

Structure with XML Tags

XML Tags for Prompt Structure - Using XML tags to organize and delineate different content sections within prompts to improve AI comprehension.

Purpose - When interpolating large amounts of content into prompts, XML tags help AI models distinguish between different types of information and understand text grouping.

Implementation - Wrap content sections in descriptive XML tags like

<sales_records></sales_records> or <my_code></my_code> rather than dumping unstructured text.

Tag naming - Use descriptive, specific tag names (e.g., "sales_records" better than "data") to provide context about content nature.

Example use case - Debugging prompt with mixed code and documentation becomes clearer when separated into `<my_code>` and `<docs>` tags.

Benefits - Makes prompt structure obvious to AI, reduces confusion about content boundaries, improves output quality even for smaller content blocks.

Application - Can wrap any interpolated content like `<athlete_information>` even when content is short, to clarify it's external input requiring consideration.

Providing Examples

One-shot/Multi-shot prompting - providing examples in prompts to guide model behavior. One-shot = single example, multi-shot = multiple examples.

Implementation: Structure examples with XML tags containing sample input and ideal output. Always wrap examples clearly to distinguish from actual prompt content.

Key applications:

- Corner case handling (sarcasm detection, edge scenarios)
- Complex output formatting (JSON structures, specific formats)
- Clarifying expected response quality/style

Best practices:

- Add context for corner cases ("be especially careful with sarcasm")
- Include reasoning explaining why output is ideal
- Use highest-scoring examples from prompt evaluations as templates
- Place examples after main instructions/guidelines

Effectiveness boost: Combine examples with explanations of what makes them ideal to reinforce desired output characteristics.

MODULE 3

Tool Use and Structured Outputs

Introducing Tool Use

Tool use - method for Claude to access external information beyond training data.

Default limitation: Claude only knows information from training data, lacks current/real-time information.

Tool use flow:

25. Send initial request to Claude + instructions for external data access
26. Claude evaluates if external data needed, requests specific information
27. Server runs code to fetch requested data from external sources
28. Send follow-up request to Claude with retrieved data
29. Claude generates final response using original prompt + external data

Weather example: User asks current weather -> Claude requests weather data -> Server calls weather API -> Claude receives weather data -> Claude provides informed weather response.

Key concept: Tools enable Claude to augment responses with live/current information by orchestrating external data retrieval between Claude's requests.

Project Overview

Project Overview

Goal - Teach Claude to set time-based reminders through tool implementation in Jupyter notebook

Target interaction - User: "Set reminder for doctor's appointment, week from Thursday" -> Claude: "I will remind you at that point in time"

Three core problems requiring tools:

30. Time knowledge gap = Claude knows current date but not exact time
31. Time calculation errors = Claude sometimes miscalculates time-based addition (e.g., 379 days from January 13th, 1973)
32. No reminder mechanism = Claude understands reminder concept but lacks implementation capability

Three corresponding tools to build:

33. Current datetime tool = Gets current date + time
34. Duration addition tool = Adds time duration to datetime (e.g., current date + 20 days)
35. Reminder setting tool = Actually sets the reminder

Implementation approach - One tool at a time, building toward multi-tool coordination

Tool Functions

Tool Functions - Python functions executed automatically when Claude needs extra information to help users.

Key characteristics:

- Plain Python functions called by Claude when it determines additional data is needed
- Must use descriptive function names and argument names
- Should validate inputs and raise errors with meaningful messages
- Error messages are visible to Claude, allowing it to retry with corrected parameters

Best practices:

36. Well-named functions and arguments
37. Input validation with immediate error raising for invalid inputs
38. Meaningful error messages that guide correction

Example implementation pattern:

```
\\
def get_current_datetime(date_format="%Y%m%d %H:%M:%S"):
if not date_format:
raise ValueError("date format cannot be empty")
return datetime.now().strftime(date_format)
\\
```

Tool function workflow: Claude identifies need for information -> calls tool function -> receives result or error -> may retry with corrections if error occurred.

Purpose: Extend Claude's capabilities beyond its training data by providing access to real-time information like current datetime, weather, etc.

Tool Schemas

Tool Schemas - JSON schema specifications that describe tool functions and their parameters for language models

JSON Schema - data validation specification (not ML-specific) used to validate JSON data, adopted by ML community for tool calling

Tool Schema Structure:

- name: tool identifier
- description: 3-4 sentences explaining what tool does, when to use, what data it returns
- input_schema: actual JSON schema describing function arguments with types and descriptions

Schema Generation Trick:

39. Take tool function to Claude.ai
40. Prompt: "write valid JSON schema spec for tool calling for this function, follow best practices in attached documentation"
41. Attach Anthropic API documentation tool use page
42. Copy generated schema

Implementation Pattern:

- Name functions descriptively
- Name schemas as [function_name]_schema
- Import ToolParam from anthropic.types
- Wrap schema dictionary with ToolParam() to prevent type errors

Purpose - inform Claude about available tools, required arguments, and usage context through standardized JSON validation format

Handling Message Blocks

Tool-Enabled Claude Requests

Step 3: Making requests to Claude with tools - include tool schema in request alongside user message using `\tools\` keyword argument containing JSON schema specs.

Multi-Block Messages

Content structure change - messages now contain multiple blocks instead of just text blocks.

Tool response format - assistant message with:

- Text block = user-facing explanation
- Tool use block = contains function name + arguments for tool execution

Message History Management

Critical requirement - manually maintain conversation history since Claude stores nothing.

Multi-block handling - append entire `response.content` (all blocks) to messages list, not just text.

Helper function updates needed - `add_user_message` and `add_assistant_message` functions must support multiple blocks instead of single text blocks only.

Conversation flow - user message -> assistant response with tool use block -> execute tool -> respond back to Claude with full history.

Sending Tool Results

Tool Results - Results from executed tool functions sent back to Claude in follow-up requests.

Process: Execute tool function requested by Claude -> Create tool result block -> Send follow-up request with full conversation history.

Tool Result Block Structure:

- `tool_use_id` = Matches ID from original tool use block to pair requests with results
- `content` = Tool function output converted to string (usually JSON)
- `is_error` = Boolean flag for function execution errors (default false)

Tool Use ID Purpose - Links multiple tool requests to correct results when Claude makes simultaneous tool calls. Each tool use gets unique ID, tool results must reference matching IDs.

Follow-up Request Requirements:

- Include complete message history (original user message + assistant tool use message + new user message with tool result)
- Must include original tool schemas even if not using tools again
- Tool result block goes in user message, not assistant message

Conversation Flow: User request -> Claude assistant response (text + tool use blocks) -> Server executes tool -> User message with tool result block -> Claude final response with integrated results.

Multi-Turn Conversations with Tools

Multi-Turn Tool Conversations - conversations where Claude uses multiple tools sequentially to answer a single user query.

Tool Chaining Process - user asks question -> Claude requests first tool -> tool executed -> result returned -> Claude requests second tool -> tool executed -> result returned -> Claude provides final answer.

Example Flow - user asks "what day is 103 days from today" -> Claude calls `get_current_datetime` -> Claude calls `add_duration_to_datetime` -> Claude provides answer.

Implementation Pattern - while loop that continues calling Claude until no more tool requests, checking each response for `tool_use` blocks.

run_conversation Function - takes initial messages, loops through Claude calls, executes requested tools, adds results to conversation, continues until final response.

Required Refactors:

- `add_user_message/add_assistant_message` = updated to handle multiple message blocks instead of just plain text
- `chat function` = accepts `tools` parameter, returns entire message instead of just first text block
- `text_from_message helper` = extracts all text blocks from a message with multiple content blocks

Key Insight - can't predict how many tools user queries will require, so system must handle arbitrary chains of tool calls automatically.

Implementing Multiple Turns

Multiple Turns Implementation = continuously calling Claude until it stops requesting tools

Stop Reason Field = indicates why Claude stopped generating text

- `stop_reason` = "tool_use" means Claude wants to call a tool
- Other values exist but `tool_use` is most commonly checked

run_conversation Function = main loop that:

43. Calls Claude with messages + available tools
44. Adds assistant response to conversation history
45. Checks `stop_reason` - if not "tool_use", breaks loop
46. If `tool_use`, calls `run_tools` function
47. Adds tool results as user message
48. Repeats until no more tool requests

run_tools Function = processes multiple tool use blocks:

49. Filters `message.content` for blocks with `type="tool_use"`
50. Iterates through each tool request
51. Runs appropriate tool function via `run_tool` helper
52. Creates `tool_result` blocks with: `type="tool_result"`, `tool_use_id=original_id`, `content=JSON_encoded_output`, `is_error=boolean`
53. Returns list of all tool result blocks

run_tool Function = dispatcher that:

- Takes tool_name and tool_input
- Uses if statements to match tool names to functions
- Executes appropriate tool function
- Scalable for adding multiple tools

Error Handling = try/except blocks around tool execution:

- Success: is_error=false, content=tool_output
- Failure: is_error=true, content=error_message

Key Architecture Points:

- Assistant messages can contain multiple blocks (text + multiple tool_use)
- Each tool_use block gets separate tool_result response
- Tool results sent back as user message containing all results
- Process repeats until Claude provides final text-only response

Using Multiple Tools

Multiple Tools Implementation - Adding additional tools to an existing tool system after initial framework setup.

Process - 3 steps: (1) Add tool schemas to RunConversation function's tools list, (2) Add conditional cases in RunTool function to handle new tool names, (3) Implement actual tool functions.

Key Components:

- RunConversation function = Contains tools list that makes Claude aware of available tools
- RunTool function = Routes tool calls to appropriate functions based on tool name
- Tool schemas = Define tool structure for the AI model
- Tool functions = Actual implementation code

Example Tools Added:

- AddDurationToDateTime = Calculates date/time with duration offset
- SetReminder = Creates reminder (mock implementation that prints confirmation)

Tool Chaining - AI can use multiple tools sequentially in single conversation (e.g., calculate date first, then set reminder with result).

Message Structure - Assistant responses can contain multiple blocks: text blocks + tool use blocks in same message.

Scalability - After initial framework setup, adding new tools becomes simple pattern of schema + routing + implementation.

The Batch Tool

Batch Tool - tool that enables Claude to run multiple tools in parallel within a single Assistant message instead of making separate sequential requests.

Problem: Claude can technically send multiple tool use blocks in one message but rarely does so in practice, leading to unnecessary sequential tool calls.

Solution: Create batch tool schema that takes list of invocations (each containing tool name + arguments). Instead of calling tools directly, Claude calls batch tool with array of desired tool executions.

Implementation:

- Add batch tool to schema with invocations parameter
- Create `run_batch` function that iterates through invocations list
- Extract tool name and JSON-parsed arguments from each invocation
- Call `run_tool` function for each requested tool
- Return `batch_output` list containing results from all tool executions

Mechanism: Tricks Claude into parallel tool execution by providing higher-level abstraction that manually handles what multiple tool use blocks would accomplish automatically.

Result: Single request-response cycle instead of multiple sequential rounds for parallel-executable tasks.

Tools for Structured Data

Tools for Structured Data - alternative method to extract structured JSON from data sources using Claude's tool system instead of message pre-fill and stop sequences.

Key differences from prompt-based extraction:

- More reliable output
- More complex setup
- Requires JSON schema specification

Core Process:

54. Define JSON schema for tool where inputs = desired data structure
55. Send prompt + schema to Claude
56. Claude calls tool with structured arguments matching schema
57. Extract JSON from tool use block (no tool result needed)

Critical requirement - Force tool calling using `tool_choice` parameter:

- `tool_choice = {"type": "tool", "name": "your_tool_name"}`
- Ensures Claude always calls specified tool

Implementation steps:

58. Create schema definition for extraction tool
59. Update chat function to accept `tool_choice` parameter
60. Pass `tool_choice` to `client.messages.create()`
61. Access structured data from `response.content[0].input`

Use cases - When reliability more important than simplicity. Prompt-based methods better for quick/simple extractions, tools better for complex/reliable extractions.

Fine Grained Tool Calling

Tool Streaming - streaming API responses while using tools with Claude

Key Components:

- Standard streaming returns `content_block_delta` events
- Tool streaming adds `input_json_delta` events with `partial_json` (chunk) and `snapshot` (cumulative sum)
- Implementation requires handling additional event type in streaming pipeline

Fine-Grained Tool Calling - feature that disables JSON validation for faster streaming

Default Behavior:

- Claude generates JSON chunks for tool arguments
- API buffers chunks until complete top-level key-value pair is generated
- Validates JSON against schema before sending chunks to server
- Results in delays followed by burst of chunks arriving simultaneously

Fine-Grained Mode (`fine_grained: true`):

- Disables API-side JSON validation
- Sends chunks immediately as generated
- Provides traditional streaming experience
- Requires client-side error handling for invalid JSON

Trade-offs:

- Default = slower but validated JSON
- Fine-grained = faster streaming but potential invalid JSON (like "undefined" instead of null)
- Invalid JSON in default mode gets wrapped as string rather than proper object structure

Use Cases:

- Fine-grained useful for immediate UI updates or early processing of tool arguments
- Default sufficient when validation delays acceptable

MODULE 4

Built-in Tools, Search, RAG, and Multimodal Input

The Text Edit Tool

Text Editor Tool - built-in Claude tool for file/text operations (read, write, create, replace, undo files/directories)

Key characteristics:

- Only JSON schema built into Claude, implementation must be custom-coded
- Schema stub sent to Claude gets auto-expanded to full schema
- Schema type string varies by Claude model version (3.5 vs 3.7 have different dates)
- Enables Claude to act as software engineer out-of-the-box

Required implementation:

- Custom class/functions to handle Claude's tool use requests
- Functions for: view files, string replace, create files, etc.
- Actual file system operations not provided by Claude

Workflow:

62. Send minimal schema stub to Claude (name + type with version-specific date)
63. Claude expands to full schema internally
64. Claude sends tool use requests
65. Custom implementation executes actual file operations
66. Results sent back to Claude

Use cases:

- Replicate AI code editor functionality
- File system operations where native editors unavailable
- Automated code generation/refactoring
- Multi-file project manipulation

Benefits - approximates fancy code editor capabilities through API calls rather than GUI interaction.

The Web Search Tool

Web Search Tool - built-in Claude tool for searching web to find up-to-date/specialized information for user questions

Implementation - no custom code needed, Claude handles search execution automatically

Schema Requirements:

- type: "web_search_20250305"
- name: "web_search"
- max_uses: number (limits total searches, default 5)
- allowed_domains: optional list to restrict search to specific domains

Response Structure:

- Text blocks = Claude's explanatory text
- Tool use blocks = search queries Claude executed
- Web search result blocks = found pages (title, URL)
- Citation blocks = specific text supporting Claude's statements

Key Features:

- Multiple searches possible per request (up to max_uses limit)
- Domain restriction available for quality control
- Citation system links statements to source material

UI Rendering Pattern:

- Display text blocks as normal text
- Show search results as reference list
- Highlight citations with source attribution (domain, title, URL, quoted text)

Use Case Example: Restricting to NIH.gov for medical/exercise advice ensures scientifically-backed information vs generic web content.

Introducing Retrieval Augmented Generation

RAG - Retrieval Augmented Generation technique for querying large documents using language models.

Problem: How to extract specific information from large documents (100-1000+ pages) using Claude without hitting context limits.

Option 1 (Direct approach): Place entire document text directly into prompt.

- Limitations: Hard token limits, decreased effectiveness with longer prompts, higher costs, slower processing

Option 2 (RAG approach): Two-step process

- Step 1: Break document into small chunks
- Step 2: For user questions, find most relevant chunks and include only those in prompt

RAG benefits: Model focuses on relevant content, scales to large/multiple documents, smaller prompts, lower costs, faster processing

RAG downsides: More complexity, requires preprocessing, needs search mechanism to find relevant chunks, no guarantee chunks contain complete context, multiple chunking strategies possible (equal portions vs header-based)

Key challenge: Defining relevance and optimal chunking strategy for specific use cases.

RAG trades simplicity for scalability and efficiency but requires careful implementation and evaluation.

Text Chunking Strategies

Text Chunking Strategies - process of dividing documents into smaller pieces for RAG pipelines

Core Problem: Chunking quality directly impacts RAG performance. Poor chunking leads to irrelevant context retrieval (e.g., medical "bug" text retrieved for software engineering query about bugs).

Three Main Strategies:

67. Size-Based Chunking = dividing text into equal-length strings

- Pros: Easy to implement, most common in production
- Cons: Cut-off words, lacks context
- Solution: Overlap strategy = include characters from neighboring chunks to preserve context
- Trade-off: Creates text duplication but improves chunk meaning

68. Structure-Based Chunking = dividing based on document structure (headers, paragraphs, sections)

- Best for structured documents (markdown, HTML)
- Limitation: Requires guaranteed document formatting
- Example: Split on markdown headers (##) to create section-based chunks

69. Semantic-Based Chunking = using NLP to group related sentences/sections

- Most advanced technique
- Groups consecutive sentences based on semantic similarity
- Complex implementation

Key Implementation Notes:

- Chunk by character = most reliable fallback, works with any document type
- Chunk by sentence = good middle ground if sentence detection works reliably
- Chunk by section = optimal results but requires structured input
- Strategy choice depends on document type guarantees and use case requirements

Rule: No universal best chunking method - depends on document structure guarantees and specific use case.

Text Embeddings

Text Embeddings - numerical representation of text meaning generated by embedding models

Embedding Model - takes text input, outputs long list of numbers (range -1 to +1)

Embedding Numbers - scores representing unknown qualities/features of input text. Each number theoretically scores different aspects (happiness, topic relevance, etc.) but actual meaning is unknown to users.

Semantic Search - uses text embeddings to find text chunks related to user questions in RAG pipelines. Solves the search problem of matching user queries to relevant document chunks.

RAG Pipeline Process - extract text chunks -> user submits query -> find related chunks using semantic search -> add relevant chunks as context to prompt

Implementation - Anthropic recommends Voyage AI for embedding generation. Requires separate account/API key. Free to start, easy integration via SDK.

Key Insight - Embeddings enable semantic similarity matching rather than keyword matching, allowing better understanding of text relationships for retrieval tasks.

The Full RAG Flow

RAG Flow - 7-step process combining text chunking, embeddings, and vector search to retrieve relevant context for LLM queries.

Step 1: Text Chunking - Split source documents into separate text pieces

Step 2: Generate Embeddings - Convert text chunks into numerical vectors using embedding models

Step 3: Normalization - Scale vector magnitudes to 1.0 (handled automatically by embedding APIs)

Step 4: Vector Database Storage - Store embeddings in specialized database optimized for numerical vector operations

Step 5: Query Processing - Convert user question into embedding using same model

Step 6: Similarity Search - Find most similar stored embeddings using cosine similarity calculation

Step 7: Prompt Assembly - Combine user question with retrieved relevant text chunks, send to LLM

Key Math Concepts:

- Cosine Similarity = cosine of angle between vectors, returns values -1 to 1, closer to 1 means more similar
- Cosine Distance = 1 minus cosine similarity, values closer to 0 mean higher similarity
- Vector Database = performs similarity calculations to find closest matching embeddings

Process Flow: Pre-processing (steps 1-4) -> User Query -> Real-time retrieval (steps 5-7) -> LLM Response

Implementing the Rag Flow

RAG Flow Implementation - practical walkthrough of 5-step retrieval-augmented generation process

Step 1: Text Chunking - split document into sections using `chunk_by_section` function on `report.MD` file

Step 2: Embedding Generation - create vector representations for each chunk using `generate_embedding` function (supports single string or list of strings input)

Step 3: Vector Store Population = create vector index instance, loop through chunk-embedding pairs using `zip()`, store each pair with `store.add_vector(embedding, {content: chunk})`. Store original text with embeddings for meaningful retrieval results.

Step 4: Query Processing - user asks question "what did software engineering department do last year", generate embedding for user query

Step 5: Similarity Search - use `store.search(user_embedding, 2)` to find 2 most relevant chunks, returns results with cosine distances (0.71 for section two, 0.72 for methodology section)

Key Components:

- Vector Index Class = custom vector database implementation
- Cosine Distance = similarity metric between query and stored embeddings
- Metadata Storage = storing original text content alongside embeddings enables meaningful retrieval

Workflow complete but has limitations requiring further improvements.

BM25 Lexical Search

BM25 - Best Match 25, a lexical search algorithm commonly used in RAG pipelines to complement semantic search.

Problem with semantic search alone - Can miss exact term matches, returning irrelevant results even when specific terms appear frequently in certain documents.

Hybrid search approach - Combines semantic search (embeddings/vector database) with lexical search (BM25) in parallel, then merges results for better balance.

BM25 algorithm steps:

70. Tokenize user query into separate terms (remove punctuation, split on spaces)
71. Count frequency of each term across all text chunks/documents
72. Assign relative importance to terms based on usage frequency (rare terms = higher importance, common terms like "a" = lower importance)
73. Rank text chunks by how often they contain higher-weighted terms

Key insight - Frequently used terms across corpus are less important for search relevance than rare, specific terms.

BM25 advantages - Better at finding exact term matches, prioritizes documents containing rare/specific search terms, complements semantic search weaknesses.

Implementation - Both semantic and lexical search systems use similar APIs (add_document, search functions) making them easy to combine.

Next step - Merge results from both search systems to get benefits of semantic understanding plus exact term matching.

A Multi-Index Rag Pipeline

Multi-Index RAG Pipeline - system combining semantic search (vector index) and lexical search (BM25 index) for improved retrieval accuracy.

Key Components:

- Vector Index = semantic similarity search using embeddings
- BM25 Index = lexical/keyword-based search
- Retriever Class = wrapper that forwards queries to both indexes and merges results

Reciprocal Rank Fusion - technique for merging search results from different indexes. Formula: $RRF_score = \text{sum of } (1/(\text{rank} + 1))$ across all search methods for each document. Documents ranked by highest combined score.

Example: Vector search returns [doc2, doc7, doc6], BM25 returns [doc6, doc2, doc7]. After RRF calculation, final ranking becomes [doc2, doc6, doc7] because doc2 ranked high in both methods.

Benefits:

- Improved search accuracy by combining different search paradigms
- Modular design with standardized API (search() and add_document() methods)
- Easy to extend with additional search indexes
- Better handling of edge cases where single method fails

Implementation pattern allows multiple search methodologies to work together while maintaining separate, isolated index classes.

Reranking Results

Reranking - post-processing step that uses LLM to reorder search results by relevance after initial retrieval.

Process: Run vector + BM25 search -> merge results -> pass to LLM with prompt asking to rank documents by relevance -> get reordered results.

Implementation details: Use document IDs instead of full text for efficiency. LLM receives user query + candidate documents + instruction to return most relevant docs in decreasing order. Assistant message pre-fill + stop sequence ensures structured JSON output.

Tradeoffs: Increases search accuracy by leveraging LLM's understanding of semantic relevance. Increases latency due to additional LLM call. Particularly effective when initial retrieval methods miss nuanced query intent (e.g., "ENG team" vs "engineering team").

Example improvement: Query "What did engineering team do with incident 2023?" correctly prioritized software engineering section over cybersecurity section after reranking, despite hybrid search initially ranking it lower.

Contextual Retrieval

Contextual Retrieval - technique to improve RAG pipeline accuracy by adding context to document chunks before embedding.

Problem: When documents are split into chunks, individual chunks lose context from the original document, reducing retrieval accuracy.

Solution: Pre-processing step that adds contextual information to each chunk before inserting into retriever database.

Process:

74. Take individual chunk + original source document
75. Send to LLM (Claude) with prompt asking to generate situating context
76. LLM generates brief context explaining chunk's relationship to larger document
77. Join generated context with original chunk = "contextualized chunk"
78. Use contextualized chunk as input to vector/BM25 indexes

Large Document Handling: If source document too large for single prompt, use selective context strategy:

- Include starter chunks (1-3) from document beginning for summary/abstract
- Include chunks immediately before target chunk for local context
- Skip middle chunks that provide less relevant context

Implementation: add_context function takes text chunk + source text, generates context via LLM, concatenates context with original chunk, returns contextualized version.

Benefit: Chunks retain ties to larger document structure and cross-references, improving retrieval accuracy for complex documents with interconnected sections.

Extended Thinking

Extended Thinking - Claude feature that allows reasoning time before generating final response

Key mechanics:

- Displays separate thinking process visible to users
- Increases accuracy for complex tasks but adds cost (charged for thinking tokens) and latency
- Thinking budget = minimum 1024 tokens allocated for thinking phase
- Max tokens must exceed thinking budget (e.g., budget 1024 requires max_tokens >= 1025)

When to use:

- Enable after prompt optimization fails to achieve desired accuracy
- Use prompt evals to determine necessity

Response structure:

- Thinking block = contains reasoning text + cryptographic signature
- Text block = final response
- Signature = prevents tampering with thinking text (safety measure)

Special cases:

- Redacted thinking blocks = encrypted thinking text flagged by safety systems
- Provided for conversation continuity without losing context
- Can force redacted blocks using test string: "entropic magic string triggered redacted thinking [special characters]"

Implementation:

- Set thinking=true and thinking_budget parameter
- Ensure max_tokens > thinking_budget for adequate response generation capacity

Image Support

Claude Vision Capabilities - ability to process images within user messages for analysis, comparison, counting, and description tasks.

Image Limitations:

- Max 100 images per request
- Size/dimension restrictions apply
- Images consume tokens (charged based on pixel height/width calculation)

Image Block Structure - special block type within user messages that holds either raw image data (base64) or URL reference to online image. Multiple image blocks allowed per message.

Critical Success Factor - strong prompting techniques required for accurate results. Simple prompts often fail.

Prompting Techniques for Images:

- Step-by-step analysis instructions
- One-shot/multi-shot examples (alternating image and text pairs)
- Clear guidelines and verification steps
- Structured analysis frameworks

Example Use Case - automated fire risk assessment from satellite imagery analyzing tree density, property access, roof overhang, and assigning numerical risk scores.

Implementation - base64 encode image data, create message with image block (type: image, source: base64, media_type, data) followed by text block containing detailed prompt instructions.

Key Takeaway - image accuracy depends entirely on prompt sophistication, not just image quality.

PDF Support

PDF Support in Claude:

Claude can read PDF files directly using similar code to image processing.

Key implementation changes:

- File type = "document" instead of "image"
- Media type = "application/pdf" instead of "image/png"
- Variable naming = file_bytes instead of image_bytes

Claude PDF capabilities - read text + images + charts + tables + mixed content extraction

PDF processing - one-stop solution for comprehensive document analysis

Usage pattern - same as image input but with document-specific parameters

Citations

Citations - feature allowing Claude to reference source documents and show where information comes from

Citation types:

- citation_page_location = for PDF documents, shows document index/title/start page/end page/cited text
- citation_char_location = for plain text, shows character position in text block

Implementation:

- Add "citations": {"enabled": true} to request
- Add "title" field to identify source document
- Works with both PDF files and plain text sources

Response structure - content becomes list of text blocks, some containing citations arrays with location data

Purpose - transparency for users to verify Claude's information sources and check accuracy of interpretations

UI benefit - enables citation popups/overlays showing source document, page numbers, and exact cited text when users hover over referenced content

Key use case - ensuring users can investigate how Claude builds responses from source materials rather than appearing to speak from memory alone

Performance, Files, and Code Execution

Prompt Caching

Prompt Caching - feature that speeds up Claude's responses and reduces text generation costs by reusing computational work from previous requests.

Normal request flow: User sends message -> Claude processes input (creates internal data structures, performs calculations) -> Claude generates output -> Claude discards all processing work -> Ready for next request.

Problem: When follow-up requests contain identical input messages, Claude must repeat all the same computational work it just threw away, creating inefficiency.

Solution: Prompt caching stores the results of input message processing in temporary cache instead of discarding. When identical input appears in subsequent requests, Claude retrieves cached work rather than reprocessing, dramatically speeding response generation.

Key benefit: Reuses previous computational work to avoid redundant processing of repeated content.

Rules of Prompt Caching

Prompt Caching - system that saves processing work from initial request to reuse in follow-up requests with identical content

Core mechanism: Initial request -> Claude processes + saves work to cache -> Follow-up requests with identical content -> Claude retrieves cached work instead of reprocessing

Cache duration - 1 hour maximum

Cache activation requires manual cache breakpoint addition to message blocks

Text block formats:

- Shorthand: content = "text string" (cannot add cache control)
- Longhand: content = [{"type": "text", "text": "content", "cache_control": {...}}] (required for caching)

Cache scope - all content up to and including breakpoint gets cached

Cache invalidation - any change in content before breakpoint invalidates entire cache

Content processing order - tools -> system prompt -> messages (joined together)

Cache breakpoint placement options:

- Tool schemas
- System prompts
- Message blocks (text, image, tool use, tool result)

Maximum breakpoints - 4 per request

Multiple breakpoints - create multiple cache layers, partial cache hits possible if only later content changes

Minimum cache threshold - 1024 tokens required for content to be cached

Best use cases - repeated identical content (system prompts, tool definitions, static message prefixes)

Prompt Caching in Action

Prompt Caching Implementation - automatically caches tool schemas and system prompts to reduce token usage

Setup - modify chat function to enable caching by default for tools and system prompts

Tool Schema Caching - add `cache_control` field with type "ephemeral" to last tool in list. Best practice: create copy of tools list, clone last tool schema, add cache control, then overwrite to avoid modifying original schemas

System Prompt Caching - wrap system prompt in text block dictionary with `cache_control` type "ephemeral"

Multiple Cache Breakpoints - can set cache points for both tools and system prompt in single request

Cache Order - tools -> system prompt -> messages

Token Usage Patterns:

- `cache_creation_input_tokens` = tokens written to cache on first use
- `cache_read_input_tokens` = tokens retrieved from cache on subsequent identical requests
- Partial cache reads possible when some content matches cached data

Cache Invalidation - any change to cached content (tools or system prompt) invalidates cache, forces new cache creation

Use Cases - identical content across requests - same tool schemas, system prompts, or message sequences

Code Execution and the Files API

Files API - allows uploading files ahead of time and referencing them later via file ID instead of including raw file data in each request. Upload file -> get file metadata object with ID -> use ID in future requests.

Code Execution = server-based tool where Claude executes Python code in isolated Docker containers. No implementation needed, just include predefined tool schema. Claude can run code multiple times, interpret results, generate final response.

Key constraints: Docker containers have no network access. Data input/output relies on Files API integration.

Combined workflow: Upload file via Files API -> get file ID -> include ID in container upload block -> ask Claude to analyze -> Claude writes/executes code with access to uploaded file -> returns analysis and results.

Claude can generate files (plots, reports) inside container that can be downloaded using file IDs returned in response.

Use cases: Data analysis, file processing, automated code generation for complex tasks. Response contains code blocks, execution results, and final analysis.

Implementation: Use container upload block with file ID, include analysis prompt, Claude handles code execution automatically.

MODULE 6

Model Context Protocol (MCP)

Introducing MCP

MCP - Model Context Protocol, communication layer providing Claude with context and tools without requiring developers to write tedious code.

Architecture: MCP client connects to MCP server. Server contains tools, resources, and prompts as internal components.

Problem solved: Eliminates burden of authoring/maintaining numerous tool schemas and functions for service integrations. Example: GitHub chatbot would require implementing tools for repositories, pull requests, issues, projects - significant developer effort.

Solution: MCP server handles tool definition and execution instead of your application server.

MCP servers - interfaces to outside services, wrapping functionality into ready-to-use tools.

Key benefits: Developers avoid writing tool schemas and function implementations themselves.

Common questions:

- Who creates MCP servers? Anyone, often service providers make official implementations (AWS, etc.)
- vs direct API calls? MCP eliminates need to author tool schemas/functions yourself
- vs tool use? MCP and tool use are complementary - MCP handles WHO does the work (server vs developer), both still involve tools

Core value: Shifts integration burden from application developers to MCP server maintainers.

MCP Clients

MCP Client - communication interface between your server and MCP server, provides access to server's tools

Transport agnostic - client/server can communicate via multiple protocols (stdio, HTTP, WebSockets)

Common setup - client and server on same machine using standard input/output

Communication - message exchange defined by MCP spec

Key message types:

- list tools request = client asks server for available tools
- list tools result = server responds with tool list
- call tool request = client asks server to run tool with arguments
- call tool result = server responds with tool execution result

Typical flow:

79. User queries server

80. Server requests tool list from MCP client

81. MCP client sends list tools request to MCP server

82. MCP server responds with list tools result

83. Server sends query + tools to Claude
84. Claude requests tool execution
85. Server asks MCP client to run tool
86. MCP client sends call tool request to MCP server
87. MCP server executes tool (e.g. GitHub API call)
88. Results flow back through chain: MCP server -> MCP client -> server -> Claude -> user

Purpose - enables servers to delegate tool execution to specialized MCP servers while maintaining Claude integration

Project Setup

CLI-based chatbot project - teaches MCP client-server interaction through hands-on implementation

Project components:

- MCP client = connects to custom MCP server
- MCP server = provides 2 tools (read document, update document)
- Document collection = fake documents stored in memory only

Key distinction: Normal projects implement either client OR server, not both. This project implements both for educational purposes.

Setup process:

89. Download CLI_project.zip starter code
90. Extract and open in code editor
91. Follow readme.md setup directions
92. Add API key to .env file
93. Install dependencies (with/without UV)
94. Run project: "uv run main.py" or "python main.py"
95. Test with chat prompt

Expected outcome - working chat interface that responds to basic queries, ready for MCP feature additions.

Defining Tools with MCP

MCP server implementation using Python SDK creates tools through decorators rather than manual JSON schemas.

MCP Python SDK - Official package that auto-generates tool JSON schemas from Python function definitions using @mcp.tool decorator.

Tool definition syntax - @mcp.tool(name="tool_name", description="description") + function with typed parameters using Field() for argument descriptions.

Two tools implemented:

96. read_doc_contents = Takes doc_id string, returns document content from in-memory docs dictionary
97. edit_document = Takes doc_id, old_string, new_string parameters, performs find/replace on document content

Error handling - Check if doc_id exists in docs dictionary, raise ValueError if not found.

Key advantage - SDK eliminates manual JSON schema writing, generates schemas automatically from Python function signatures and decorators.

Required imports - Field from pydantic for parameter descriptions, mcp package for server and tool decorators.

Implementation pattern - Decorator defines tool metadata, function parameters define tool arguments with types and descriptions, function body contains tool logic.

The Server Inspector

MCP Inspector - in-browser debugger for testing MCP servers without connecting to applications

Access: Run `\mcp dev [server_file.py]` in terminal -> opens server on port -> navigate to provided URL in browser

Interface: Left sidebar has connect button -> top menu shows resources/prompts/tools sections -> tools section lists available tools -> click tool to open right panel for manual testing

Testing workflow: Connect to server -> navigate to tools -> select specific tool -> input required parameters -> click run tool -> verify output

Key features: Live development testing, manual tool invocation, parameter input forms, success/failure feedback, no need for full application integration

Note: UI actively changing during development, core functionality remains similar

Example usage: Test document tools by inputting document IDs, verify read operations, test edit operations, chain operations to verify changes

Primary benefit: Debug MCP server implementations efficiently during development phase

Implementing a Client

MCP Client Implementation:

MCP Client - wrapper class around client session for resource cleanup and connection management to MCP server

Client Session - actual connection to MCP server from MCP Python SDK, requires resource cleanup on close

Client Purpose - exposes MCP server functionality to rest of codebase, enables reaching out to server for tool lists and tool execution

Key Functions:

- `list_tools()` = `await self.session.list_tools()`, return `result.tools`
- `call_tool()` = `await self.session.call_tool(tool_name, tool_input)`

Usage Flow - client gets tool definitions to send to Claude, then executes tools when Claude requests them

Common Pattern - wrap client session in larger class for resource management rather than use session directly

Testing - can run client file directly with testing harness to verify server connection and tool retrieval

Integration - other code in project calls client functions to interact with MCP server, enabling Claude to inspect/edit documents through defined tools

Defining Resources

MCP Resources - mechanism allowing MCP servers to expose data to clients for read operations

Resource Types - 2 types: direct (static URI like "docs://documents") and templated (parameterized URI like "docs://documents/{doc_id}")

URI - address/identifier for accessing specific resource, defined when creating resource

Resource Flow - client sends read resource request with URI -> server matches URI to function -> server executes function -> returns data in read resource result

Implementation - use `@mcp.resource` decorator with URI and MIME type parameters

MIME Types - hint to client about returned data format (application/json for structured data, text/plain for plain text)

Templated Resources - URI parameters automatically parsed by SDK and passed as keyword arguments to handler function

Resource vs Tools - resources provide data proactively (fetch document contents when @ mentioned), tools perform actions reactively (when Claude decides to call them)

Data Return - SDK automatically serializes returned data to strings, client responsible for deserialization

Testing - MCP inspector can list direct resources separately from templated resources, allows testing individual resource calls

Accessing Resources

MCP Resource Access Implementation:

Resource Reading Function - client-side function to request and parse resources from MCP server

Function Parameters - URI (resource identifier)

Implementation Steps:

- Import json module + AnyURL from pydantic
- Call `await self.session.read_resource(AnyURL(uri))`
- Extract first element from `result.contents[0]`
- Check `resource.mime_type` for parsing strategy

Content Parsing Logic:

- If `mime_type == "application/json"` -> return `json.loads(resource.text)`
- Otherwise -> return `resource.text` (plain text)

Server Response Structure - `result.contents` list with first element containing type/mime_type metadata

Resource Integration - MCP client functions called by other application components to fetch document contents for prompts

End Result - Document contents automatically included in Claude prompts without requiring tool calls

Key Point - Resources expose server information directly to clients through structured request/response pattern

Defining Prompts

MCP Prompts - Pre-defined, tested prompt templates that MCP servers expose to client applications for specialized tasks.

Purpose - Instead of users writing ad-hoc prompts, server authors create high-quality, evaluated prompts tailored to their server's domain.

Implementation - Use `@mcpserver.prompt` decorator with name/description, define function that returns list of messages (user/assistant messages that can be sent directly to Claude).

Example Use Case - Document formatting prompt that takes document ID, instructs Claude to read document using tools, reformat to markdown, and save changes.

Key Benefits - Server-specific expertise, pre-tested quality, reusable across client applications, better results than user-generated prompts.

Message Structure - Returns `base.UserMessage` objects containing the formatted prompt text with interpolated parameters.

Client Integration - Prompts appear as autocomplete options (slash commands) in client applications, prompt user for required parameters, then execute the pre-built prompt workflow.

Prompts in the Client

MCP Client Prompt Implementation:

List prompts - `await self.session.list_prompts()`, return `result.prompts`

Get prompt - `await self.session.get_prompt(prompt_name, arguments)`, return `result.messages`

Prompt workflow:

98. Define prompt in MCP server with expected arguments (e.g., `document_id`)
99. Client calls `get_prompt` with prompt name + arguments dictionary
100. Arguments passed as keyword arguments to prompt function
101. Function interpolates arguments into prompt text
102. Returns messages array for direct feeding to LLM

Key concept: Prompts are server-defined templates that clients can invoke with specific arguments to generate contextualized instructions for LLMs. Arguments flow from client call -> prompt function -> interpolated prompt text -> LLM consumption.

MODULE 7

Claude Code and Computer Use

Anthropic Apps

Anthropic Apps - two deployed applications by Anthropic: Claude Code and Computer Use.

Claude Code - terminal-based coding assistant that serves as example of agent architecture.

Computer Use - toolset that expands Claude's capabilities beyond text generation.

Key purpose - these apps demonstrate agent concepts and provide practical examples for understanding agent design and implementation.

Setup process - involves terminal configuration for Claude Code usage on sample projects.

Agent connection - both applications exemplify how agents work, serving as learning models for building effective agents.

Claude Code Setup

Claude Code - terminal-based coding assistant program that helps with code-related tasks

Core capabilities - search/read/edit files + advanced tools (web fetching, terminal access) + MCP client support for expanded functionality via MCP servers

Setup process:

103. Install Node.js (check with "npm help" command)
104. Run npm install to install Claude Code
105. Execute "claude" command in terminal to login to Anthropic account

Full setup guide - docs.anthropic.com

MCP client functionality - can consume tools from MCP servers to extend capabilities beyond basic file operations

Claude Code in Action

Claude Code - AI coding assistant that functions as a collaborative engineer on projects, not just a code generator.

Key capabilities: project setup, feature design, code writing, testing, deployment, error fixing in production.

Setup workflow:

- Download project, open in editor
- Run `\claude\` command to launch
- Ask Claude to read README and execute setup directions
- Run `\init\` command = Claude scans codebase for architecture/coding style, creates `claude.md` file
- `claude.md` = automatically included context for future requests

Memory types: Project (shared), Local, User memory files.

Context management:

- Use # symbol to add specific notes to memory
- Can manually edit claude.md or rerun init to update
- Claude can handle Git operations (staging, committing)

Effective prompting strategies:

Method 1 - Three-step workflow:

106. Identify relevant files, ask Claude to analyze them
107. Describe feature, ask Claude to plan solution (no code yet)
108. Ask Claude to implement the plan

Method 2 - Test-driven development:

109. Provide relevant context
110. Ask Claude to suggest tests for the feature
111. Select and implement chosen tests
112. Ask Claude to write code until tests pass

Core principle: Claude Code - effort multiplier. More detailed instructions = significantly better results. Treat as collaborative engineer, not just code generator.

Enhancements with MCP Servers

Claude Code - AI assistant with embedded MCP (Model Context Protocol) client that can connect to MCP servers to expand functionality.

MCP Server Integration - Connect external tools/services to Claude Code via command: `\claude mcp add [server-name] [startup-command]`

Example Implementation - Document processing server exposing "Document Path to Markdown" tool, allowing Claude Code to read PDF/Word documents by running `\uv run main.py`

Dynamic Capability Expansion - MCP servers add new functions to Claude Code in real-time without core modifications.

Common Use Cases - Production monitoring (Sentry), project management (Jira), communication (Slack), custom development workflow tools.

Key Benefit - Significant flexibility increase for development workflows through modular server connections.

Setup Process - 1) Create MCP server with tools, 2) Add server to Claude Code with name and startup command, 3) Restart Claude Code to access new capabilities.

Parallelizing Claude Code

Parallelizing Claude Code - running multiple Claude instances simultaneously to complete different tasks in parallel

Core Problem - multiple Claude instances modifying same files simultaneously creates conflicts and invalid code

Solution - Git work trees providing isolated workspaces per Claude instance

Git Work Trees - feature creating complete project copies in separate directories, each corresponding to different Git branches

Workflow - create work tree -> assign task to Claude instance -> work in isolation -> commit changes -> merge back to main branch

Custom Commands - automating work tree creation/management through .claude/commands directory with markdown files containing prompts

Command Structure - .claude/commands/filename.md with \$ARGUMENTS placeholder for dynamic values

Parallel Execution Benefits - single developer commanding virtual team of software engineers, major productivity scaling limited only by engineer's management capacity

Merge Conflicts - Claude automatically resolves conflicts during branch merging process

Cleanup - Claude handles work tree removal after feature completion

Key Advantage - scales to unlimited parallel instances based on developer's capacity to manage simultaneous tasks

Automated Debugging

Automated Debugging - using AI (Claude) to automatically detect, analyze, and fix production errors without manual intervention.

Core Workflow:

113. GitHub Action runs daily to check production environment
114. Fetches CloudWatch logs from last 24 hours
115. Claude identifies errors, deduplicates them
116. Claude analyzes each error and generates fixes
117. Creates pull request with proposed solutions

Key Components:

- GitHub Actions for scheduling/automation
- AWS CLI for log retrieval
- Claude Code for error analysis and code fixes
- CloudWatch for production error monitoring

Benefits:

- Catches production-only errors (issues not present in development)
- Reduces manual log hunting and debugging time
- Provides context-aware fixes with explanations
- Creates reviewable pull requests for changes

Common Use Case: Configuration errors between environments (invalid model IDs, API keys, etc. that work locally but fail in production)

Implementation Requirements: Repository access, cloud logging service, AI coding assistant, CI/CD pipeline integration.

Computer Use

Computer Use - Claude's ability to interact with computer interfaces through visual observation and control actions.

Key capabilities:

- Takes screenshots of applications/browsers
- Clicks buttons, types text, navigates interfaces
- Follows multi-step instructions autonomously
- Performs QA testing and automation tasks

How it works:

- Runs in isolated Docker container environment
- User provides instructions via chat interface
- Claude observes screen visually and executes actions
- Generates reports on task completion/results

Primary use cases:

- Automated QA testing of web applications
- UI interaction testing across different scenarios
- Time-saving for repetitive computer tasks
- Bug identification through systematic testing

Setup requirement - Reference implementation available for local testing

Example workflow: User describes testing requirements -> Claude navigates to application -> Executes test cases -> Reports pass/fail results with detailed findings

How Computer Use Works

Computer use - tool system implementation allowing Claude to interact with computing environments

Tool use flow: User sends message + tool schema -> Claude responds with tool use request (ID, name, input) -> Server executes code -> Result sent back to Claude as tool result

Computer use follows identical flow:

- Special tool schema sent to Claude (small schema expands to larger structure behind scenes)
- Expanded schema includes action function with arguments: mouse move, left click, screenshot, etc.
- Claude sends tool use request
- Developers must fulfill request via computing environment (typically Docker container)
- Container executes programmatic key presses/mouse movements
- Response sent back to Claude

Key points:

- Claude doesn't directly manipulate computers
- Computer use = tool system + developer-provided computing environment
- Anthropic provides reference implementation (Docker container with pre-built mouse/keyboard execution code)
- Setup requires Docker + simple command execution
- Enables direct chat interface for testing Claude's computer use functionality

Computer use - abstraction layer where tool system handles Claude communication while Docker container handles actual computer interactions.

MODULE 8

Agents and Workflow Patterns

Agents and Workflows

Workflows and agents - strategies for handling user tasks that can't be completed by Claude in a single request.

Decision rule: Use workflows when you have precise task understanding and know exact steps sequence. Use agents when task details are unclear.

Workflow - series of calls to Claude for specific problems where steps are predetermined.

Example workflow: Image to 3D model converter

- Step 1: Claude describes uploaded image in detail
- Step 2: Claude uses CADQuery Python library to model object from description
- Step 3: Create rendering of model
- Step 4: Claude compares rendering to original image
- Step 5: If inaccurate, repeat from step 2 with feedback

This follows evaluator-optimizer pattern:

- Producer = generates output (Claude + CADQuery modeling)
- Evaluator = assesses output quality (comparison step)
- Loop continues until evaluator accepts output

Key point: Workflows are implementation patterns that other engineers have successfully used. Identifying workflow patterns doesn't automatically implement them - you still need to write the actual code.

Parallelization Workflows

Parallelization Workflows - breaking one complex task into multiple simultaneous subtasks, then aggregating results.

Example: Material selection for parts

- Instead of: One large prompt asking Claude to choose between metal/polymer/ceramic/composite with all criteria
- Use: Separate parallel requests, each evaluating one material's suitability, then final aggregation step to compare results

Structure: Input -> Multiple parallel subtasks -> Aggregator -> Final output

Benefits:

- Focus = Each subtask handles one specific analysis instead of juggling multiple considerations
- Modularity = Individual prompts can be improved/evaluated separately
- Scalability = Easy to add new subtasks without affecting existing ones
- Quality = Reduces confusion from overly complex single prompts

Key principle: Decompose complex decisions into specialized parallel analyses, then synthesize results.

Chaining Workflows

Chaining Workflows - breaking large tasks into series of distinct sequential steps rather than single complex prompt

Core concept: Instead of one massive prompt with multiple requirements, split into separate calls where each focuses on one specific subtask.

Example workflow: User enters topic -> search trending topics -> Claude selects most interesting -> Claude researches topic -> Claude writes script -> generate video -> post to social media

Key benefit: Allows AI to focus on individual tasks rather than juggling multiple constraints simultaneously

Primary use case: When Claude consistently ignores constraints in complex prompts despite repetition. Common with long prompts containing many "don't do X" requirements.

Problem scenario: Long prompt with constraints (don't mention AI, no emojis, professional tone) -> Claude violates some constraints regardless of repetition

Solution: Step 1 - Send initial prompt, accept imperfect output. Step 2 - Follow-up prompt asking Claude to rewrite based on specific violations found.

Critical insight: Even simple-seeming workflow becomes essential when dealing with constraint-heavy prompts that AI struggles to follow completely in single pass.

Routing Workflows

Routing Workflows - workflow pattern that categorizes user input to determine appropriate processing pipeline

Key mechanism: Initial request to Claude categorizes user input into predefined genres/categories. Based on categorization response, system routes to specialized processing pipeline with customized prompts/tools.

Example flow:

118. User enters topic (e.g., "Python functions")
119. Claude categorizes topic (e.g., "educational")
120. System uses educational-specific prompt template
121. Claude generates script with educational tone/structure

Benefits: Ensures output matches topic nature. Programming topics get educational treatment with definitions/explanations. Entertainment topics get trendy language/engaging hooks.

Structure: One routing step -> Multiple specialized processing pipelines -> Each pipeline has customized prompts/tools for specific category

Use case: Social media video script generation where different topics require different tones and approaches.

Agents and Tools

Agents - AI systems that create plans to complete tasks using provided tools, effective when exact steps are unknown. Workflows = better when precise steps are known.

Key differences: Workflows require predetermined steps, agents dynamically plan using available tools.

Agent advantages: Flexibility to solve variety of tasks with same toolset, can combine tools in unexpected ways.

Tool abstraction principle: Provide generic/abstract tools rather than hyper-specialized ones. Example - Claude code uses `bash`, `web_fetch`, `file_write` (abstract) rather than `refactor_tool`, `install_dependencies` (specialized).

Tool combination examples: `get_current_datetime` + `add_duration` + `set_reminder` can solve various time-related tasks through different combinations.

Agent behavior: Can request additional information when needed, combines tools creatively to achieve goals, works best with small set of flexible tools.

Design approach: Give agent abstract tools that can be pieced together rather than single-purpose specialized tools. This enables dynamic problem-solving and unexpected use cases.

Environment Inspection

Environment Inspection - agents evaluating their environment and action results to understand progress and handle errors.

Core concept: After each action, agents need feedback mechanisms beyond basic tool returns to understand new environment state.

Computer use example: Claude takes screenshot after every action (typing, clicking) to see how environment changed, since it cannot predict exact results of actions like button clicks.

Code editing example: Before modifying files, agents must read current file contents to understand existing state.

Social media video agent applications:

- Use Whisper CPP via `bash` to generate timestamped captions, verify dialogue placement
- Use FFmpeg to extract video screenshots at intervals, inspect visual results
- Validate video creation meets expectations before posting

Key benefit: Environment inspection enables agents to gauge task progress, detect errors, and adapt to unexpected results rather than operating blindly.

Workflows vs Agents

Workflows - pre-defined series of calls to Claude with known exact steps. Agents = flexible approach using basic tools that Claude combines to complete unknown tasks.

Key differences:

Task division: Workflows break big tasks into smaller, specific subtasks enabling higher focus and accuracy. Agents handle varied challenges creatively without predetermined steps.

Testing/evaluation: Workflows easier to test due to known execution sequence. Agents harder to test since execution path unpredictable.

User experience: Workflows require specific inputs. Agents create own inputs from user queries and can request additional input when needed.

Success rates: Workflows - higher task completion rates due to structured approach. Agents = lower completion rates due to delegated complexity.

Recommendation: Prioritize workflows for reliability. Use agents only when flexibility truly required. Users want 100% working products over fancy agents.

Core principle: Solve problems reliably first, innovation second.

Implementation Checklist

- Choose the model according to intelligence, latency, and cost requirements.
- Keep secrets server-side and build requests with model, messages, and max_tokens.
- Maintain full conversation history, including multi-block tool messages.
- Use direct prompts with explicit output requirements, XML structure, and examples where useful.
- Build an evaluation dataset and combine model grading with code-based validation.
- Implement tool schemas, dispatch, error handling, tool result blocks, and repeated tool-use loops.
- For document retrieval, combine chunking, embeddings, vector search, BM25, reranking, and contextual retrieval as needed.
- Cache stable prefixes such as tools and system prompts when repeated requests justify it.
- Use MCP to expose reusable tools, resources, and prompts through standardized clients and servers.
- Prefer workflows for predictable reliability; reserve agents for tasks with genuinely unknown execution paths.
- Inspect the environment after each action so automated systems can detect errors and adapt.